UNIVERSITY OF GHANA
DEPARTMENT OF COMPUTER SCIENCE

DCIT 204/308: Data Structures and Algorithms I & II Joint DSA Semester Project

Ghana Smart Service Operations Optimizer
A Campus Maintenance Service Dispatch System for the University of Ghana, Legon

1. Team Members:

#	Name	Index Number	Team
1	Ewura Abena Tweneboah Antwi-Bosiako	22152953	A / C —Lead & Data Structures
2	Asuako Joel Kwadwo Peprah	22185885	C — Data Structures
3	Genevieve Dzifa Gomelesio	22012665	C — Data Structures
4	Somuah Kofi Sasu	22061652	C — Data Structures
5	Nana Kwadwo Sarkodie Opoku	22026622	C — Data Structures
6	Isaac Dzikum	22045685	A- Integration and Lead assist
7	Boateng, Michelle Ama Ataa	22302392	D — Search & Sort
8	Adoboe, Kelly Kofi	22383489	B — Database
9	Agyemang, Christ Ofosu	22396432	E — Graph Engine
10	Aziamadi, Eli Makafui	22259373	C — Data Structures
11	Kwashi, Elorm Frema	22397307	D — Search & Sort
12	Essuman, Kingsley Paakwesi	22409138	E — Graph Engine
13	Akaba, Kekeli Kojo	22368056	F — Optimisation
14	Malm, Jeremy	22310005	F — Optimisation
15	Anim-Frempong, Stacy-Jane Adwoa Nyarko	22399556	B — Database

Local Ghana context: University of Ghana, Legon campus, focused on facilities/maintenance service requests across halls, lecture blocks, and administrative buildings.

Date: 23rd August, 2026

2. Problem Statement, Assumptions, Input–Output Definitions, and System Boundaries

1.1 Problem Statement

Currently, on the University of Ghana campus, maintenance requests have no systematic way to be prioritized, routed, or tracked, leading to delayed responses for urgent issues.

1.2 Assumptions

- All road/route weights are non-negative
- resource availability is updated in real time
- requests are independent of each other

1.3 Input–Output Definitions

Input	Description
Service request	Request ID, source, destination, category, urgency, status; cost/value for optimisation.
Resource	Resource ID, type, home location, capacity, availability status.
Location/road data	Location IDs/names; road endpoints and non-negative distance weights.

Output	Description
Assignment/dispatch decision	Selected resource for a request and associated distance/optimisation result.
Shortest route	Traversal order or shortest distance; unreachable nodes are explicitly reported.
Performance report	Timing/comparison data and graphs generated by the experiment module.

1.4 System Boundaries

- does not handle real-time GPS tracking
- assumes a static road network that doesn't change during a session

3. Dataset Description, Data Dictionary, and Database Schema

3.1 Dataset Description

The dataset is synthetic and was purpose-built to simulate a University of Ghana (UG), Legon campus operations management environment. It supports testing of routing, resource allocation, and service request optimization algorithms. The verified run loaded 62 locations, 138 roads, 302 service requests, and 31 resources. Only 10 nodes were reachable from node 0 because the current road dataset is disconnected.

3.2 Data Dictionary (fields in the dataset)

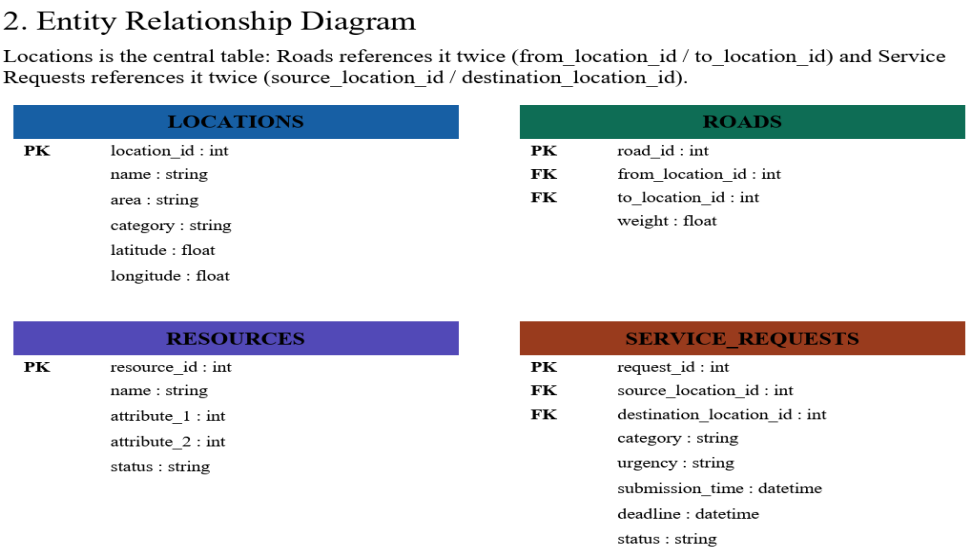
Locations: locationId, name, area, type, latitude, longitude

Roads: fromLocationId, toLocationId, distance, travelTime, roadConditionWeight

Service_requests: requestId, source, destination, category, urgency, timeSubmitted, deadline, status

Resources: resourceId, type, homeLocation, capacity, availabilityStatus

3.3 Database Schema Diagram



How the dataset was built
The four data tables were constructed as follows:

- Locations were manually curated based on publicly known University of Ghana campus facilities (hostels, lecture halls, libraries, academic departments, service points, and administrative buildings), each tagged with an area, a category, and geographic coordinates.
- Roads were simulated to represent realistic movement paths between campus locations, with distances, travel times, and condition weights generated to give graph-based algorithms (shortest path, route optimization, campus navigation) meaningful variation to work with.
- Resources (maintenance teams, security personnel, vehicles, medical support units) were defined with an availability status so resource-allocation and assignment algorithms could be tested against a realistic pool of assets.
- Service requests were generated to simulate operational incidents (maintenance issues, security reports, emergencies, facility complaints, infrastructure and connectivity issues), each carrying a source location, destination location, category, urgency level, submission time, deadline, and status.

How urgency categories were created
Each service request was assigned one of the urgency levels: Critical, High, Medium, or Low, scaled to how time-sensitive or severe the request type would realistically be. Emergency and security-related requests were weighted toward High urgency, routine operational issues toward Medium, and minor or cosmetic issues (e.g., lighting, furniture) toward Low. This gives the optimization algorithms a realistic mix of priorities to schedule and route against.

4. System Architecture and Module Design

Architecture: Database → JDBC repositories/DataLoader → AppState + Graph/custom data structures → algorithms → Console menu/output. DataLoader connects to the database, loads lists, then maps database location IDs to zero-based graph indexes and adds weighted edges. AppState retains the loaded data and graph for menu operations.

4.1 Module Ownership

Module	Responsibility	Team
db/	Schema, repositories, JDBC	B
structures/	15 hand-built data structures	C
algorithms/search/, /sort/	Search & sort	D
algorithms/graph/	BFS, DFS, Dijkstra, Prim, Kruskal	E
algorithms/optimisation/	Greedy, DP	F
App.java / integration	Console menu, wiring	Lead/A

4.2 Data Flow

At startup, the console menu waits for a choice. Loading data calls DataLoader.loadEverything(), which connects through DatabaseConnection, loads locations, roads, service requests and resources through repositories, then builds the Graph. The graph builder maps database IDs to indexes, reads road endpoints/distances and adds valid edges. The menu then passes the loaded state into search/sort, graph or optimisation operations and prints results.

5. Data Structure Implementation

4.1 Dynamic Array

A dynamic array stores elements in contiguous storage and grows when full. Index access is O(1), append is O(1) amortized, and a resize is O(n) because existing elements are copied. Middle insertion/removal is O(n). Insert a small before/after resize diagram using the final class.

4.2 Linked List

The BST stores smaller values left and larger values right. Search/insert/delete are O(h), where h is tree height; average can be O(log n), worst-case O(n). The supplied implementation supports deletion for leaf, one-child, and two-child cases and inorder traversal.

4.3 Stack

The BST stores smaller values left and larger values right. Search/insert/delete are O(h), where h is tree height; average can be O(log n), worst-case O(n). The supplied implementation supports deletion for leaf, one-child and two-child cases and inorder traversal.

4.4 Queue & Circular Queue

Queue = FIFO. CircularQueue uses an array plus front, rear and size; rear advances using modulo capacity, so freed positions are reused. enqueue/dequeue/peek are O(1). MyQueue resizes its array when needed; resizing is O(n). Show enqueue A,B,C,D, dequeue A,B, then wrap-around insertion into freed positions.

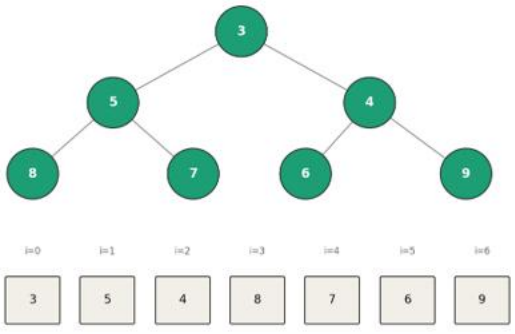
4.5 Deque

The BST stores smaller values left and larger values right. Search/insert/delete are O(h), where h is tree height; average can be O(log n), worst-case O(n). The supplied implementation supports deletion for leaf, one-child and two-child cases and inorder traversal.

4.6 Priority Queue / Heap

A priority queue always gives you the most urgent item first, no matter what order things were added in. We built ours using a binary min-heap: a tree where every parent is smaller than its children, stored efficiently in a plain array instead of using pointers.

The diagram below shows both views at once. The tree on top is just for understanding the shape; the array underneath is what's actually stored in memory. The two are connected by a simple formula: whatever index a node sits at in the array, its children sit at 2i+1 and 2i+2.

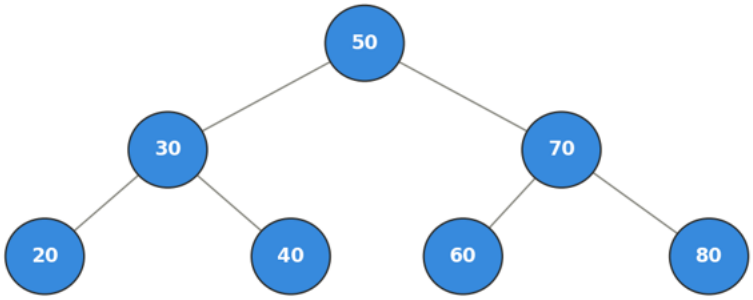


Big-O complexity

Operation	Time	Notes
Insert	O(log n)	Bubbles up at most the height of the tree
Extract minimum	O(log n)	Sinks down at most the height of the tree
Peek (view minimum)	O(1)	Always sitting at index 0
Size / is empty	O(1)	Tracked directly, no traversal needed

4.7 Binary Search Tree

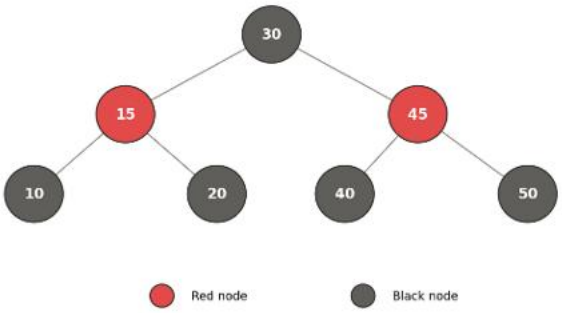
The BST stores smaller values left and larger values right. Search/insert/delete are O(h), where h is tree height; average can be O(log n), worst-case O(n). The supplied implementation supports deletion for leaf, one-child and two-child cases and inorder traversal.



Big-O complexity

Operation	Average	Worst case	Why the worst case happens
Insert	O(log n)	O(n)	Sorted or near-sorted input
Search	O(log n)	O(n)	No rebalancing when the tree grows unevenly
Delete	O(log n)	O(n)	Same as above
Inorder traversal	O(n)	O(n)	Every node must be visited regardless of shape

4.8 Red-Black Tree (or simplified balanced tree)



Big-O complexity

Operation	Time	Why it's guaranteed, not just typical
Insert	O(log n)	BST insert plus a bounded fix-up pass
Search	O(log n)	Height is mathematically capped at about 2·log2(n+1)
Delete	O(log n)	BST delete plus a bounded fix-up pass
Inorder traversal	O(n)	Every node must be visited regardless of shape

4.9 B-Tree (or index-page simulation)

The BTree uses minimum degree T=2, so a node holds at most 3 keys and 4 children. A full child is split: its middle key is promoted and the remaining keys become two children. Search/insertion are logarithmic in the tree height. Example split: [10,20,30] → left [10], promoted 20, right [30].

4.10 Hash Table

The custom HashTable uses an array of buckets with linked chains for collisions. put hashes the key, searches the chain, updates an existing key or inserts a new node, and resizes when load factor exceeds 0.75. Average get/put/remove is O(1); worst-case is O(n) with severe collisions. Load factor = size/tableSize.

4.11 Set & Map

A set stores unique values; a map associates keys with values. The supplied HashTable demonstrates the map pattern key → hash bucket → collision chain → value. Document the exact custom Set/Map implementation from the final source.

4.12 Disjoint Set / Union-Find

Union-Find maintains disjoint components. find returns a representative; union merges components. Path compression shortens find paths and union-by-rank/size attaches the smaller tree below the larger. This is especially useful in Kruskal to reject edges that would form cycles.

4.13 Graph (adjacency list & matrix)

A graph has vertices (campus locations) and edges (roads). An adjacency list stores each vertex's neighbours and weights and is efficient for sparse graphs; an adjacency matrix stores V×V edge information and uses $O(V^2)$ space. Use one small example graph and show both representations. The live project graph currently has 61 nodes and 12 roads.

5. Algorithm Implementation

For each algorithm, explain the idea, identify the key code operation, and state complexity. The following summaries are based on the supplied final/source excerpts; insert exact source snippets where the course requires them.

5.1 Linear Search / Binary Search

Linear search checks elements sequentially: O(n) worst-case, O(1) best-case, no sorted-input requirement. The supplied implementation returns -1 for null/empty arrays and not-found targets. Binary search checks the middle and halves the search interval: O(log n), but it requires sorted input consistent with compareTo().

5.2 Sorting (Selection, Insertion, Merge, Quick)

Selection Sort: $O(n^2)$ best/average/worst, in-place, generally unstable. Insertion Sort: O(n) best, $O(n^2)$ average/worst, in-place and stable. Merge Sort: O(n log n) all cases, stable, O(n) auxiliary space in the supplied implementation. Quick Sort: average O(n log n), worst $O(n^2)$ with poor pivots, in-place apart from recursion stack and not stable in the supplied form.

5.3 Graph Traversal (BFS, DFS)

BFS uses a queue and explores level by level; DFS explores deeply before backtracking. With adjacency lists both are O(V+E). The observed traversal from node 0 reached the 10 vertices in its connected component.

5.4 Shortest Path (Dijkstra)

Dijkstra starts the source at distance 0 and others at infinity, repeatedly selects the closest unsettled vertex and relaxes its outgoing edges. It is valid for non-negative edge weights. Complexity depends on implementation: simple array selection is commonly $O(V^2+E)$; heap-based implementations can be $O((V+E)\log V)$.

5.5 Minimum Spanning Tree (Prim, Kruskal)

5.6 Greedy Assignment

GreedyAssignment manually insertion-sorts requests by descending urgency, then for each request scans available resources, computes distance, selects the closest resource, marks it unavailable and records an Assignment. Equal distances are broken by lower resource ID. It is locally optimal but not globally guaranteed. Current urgency ordering is $O(R^2)$; resource scanning is O(RS) distance lookups.

5.7 Dynamic Programming (Knapsack-style selection)

Knapsack DP uses dp[i][b] = maximum value using the first i requests with budget b. If cost_i≤b: max(dp[i-1][b], value_i+dp[i-1][b-cost_i]); otherwise dp[i-1][b]. The table is O(nB) time and O(nB) space. Reconstruction walks backward and selects request i whenever dp[i][b] differs from dp[i-1][b], then subtracts its cost.

6. Correctness Evidence

6.1 Trace Tables

BINARY SEARCH TRACE TABLE

This is the trace table for the binary search algorithm using an array from the test units.

Using the sorted array [3, 7, 10, 15, 22],
our target here is 15

Step	Low	High	Mid	Mid Value	Comparison	Action
1	0	4	2	10	10 < 15	Low = 3
2	3	4	3	15	15 = 15	Found at index 3

Step-by-Step Execution Trace

Step	Extracted Node	Current Dist	Unvisited Neighbors Evaluated	Relaxation / Candidate Dist	Updated Distances `[d0, d1, d2, d3, d4]`	Predecessors `[p0, p1, p2, p3, p4]`	Min-Heap State (Vertex, Dist)
Init	–	–	–	–	`[0, ∞, ∞, ∞, ∞]`	`[-1, -1, -1, -1, -1]`	`[(0, 0.0)]`
1	**0 (Balme Library)**	0.0	1, 2, 3	d(1)=1.5, d(2)=0.8, d(3)=0.4	`[0, 1.5, 0.8, 0.4, ∞]`	`[-1, 0, 0, 0, -1]`	`[(3, 0.4), (2, 0.8), (1, 1.5)]`
2	**3 (JQB Building)**	0.4	0 (skip), 4	0.4 + 0.3 = 0.7 < ∞ (d(4)=0.7)	`[0, 1.5, 0.8, 0.4, 0.7]`	`[-1, 0, 0, 0, 3]`	`[(4, 0.7), (2, 0.8), (1, 1.5)]`
3	**4 (Dept of Physics)**	0.7	1, 3 (skip)	0.7 + 1.2 = 1.9 > 1.5 (no change)	`[0, 1.5, 0.8, 0.4, 0.7]`	`[-1, 0, 0, 0, 3]`	`[(2, 0.8), (1, 1.5)]`
4	**2 (Univ Hospital)**	0.8	0 (skip)	No unvisited neighbors	`[0, 1.5, 0.8, 0.4, 0.7]`	`[-1, 0, 0, 0, 3]`	`[(1, 1.5)]`
5	**1 (Pentagon Hostel)**	1.5	0 (skip), 4 (skip)	No unvisited neighbors	`[0, 1.5, 0.8, 0.4, 0.7]`	`[-1, 0, 0, 0, 3]`	`[]` (Empty)

INSERTION SORT TRACE TABLE

This is the trace table for the insertion sort algorithm using an array from the test units.

Using the array (12, 11, 13, 5, 6), sorting in ascending order we have

Pass	Key	Array After Pass
Initial		12 11 13 5 6
1	11	11 12 13 5 6
2	13	11 12 13 5 6
3	5	5 11 12 13 6
4	6	5 6 11 12 13

MERGE SORT TRACE TABLE

This is the trace table for the merge sort algorithm using an array from the test units.

Using the array (38, 27, 43, 3), sorting in ascending order we have

Step	Operation	Array		
Initial	Start	38 27 43 3		
Split	Left / Right	38 27	43 3	
Split	Left	38	27	
Merge	Merge left	27 38		
Split	Right	43	3	
Merge	Merge right	3 43		
Final Merge	Merge both halves	3 27 38 43		

Knapsack Dynamic Programming Trace

Example

Budget `B = 6`. Each request can be selected at most once.

Request	Cost	Value
R1	2	3
R2	3	4
R3	4	5
R4	5	8
R5	1	2

`dp[i][b]` is the best value obtainable from the first `i` requests with budget `b`.

```text
dp[i][b] = dp[i-1][b]                                  when cost[i] > b
dp[i][b] = max(dp[i-1][b], value[i] + dp[i-1][b-cost[i]]) otherwise
```

Completed Table

Requests considered	b=0	b=1	b=2	b=3	b=4	b=5	b=6
None	0	0	0	0	0	0	0
R1	0	0	3	3	3	3	3
R1-R2	0	0	3	4	4	7	7
R1-R3	0	0	3	4	5	7	8
R1-R4	0	0	3	4	5	8	8
R1-R5	0	2	3	5	6	8	**10**

For example, at `dp[5][6]`, excluding R5 gives value 8. Including R5 gives
`2 + dp[4][5] = 10`, so the algorithm stores 10.

Reconstruction

1. Start at `dp[5][6] = 10`. It differs from `dp[4][6] = 8`, so select R5 and move to budget 5.
2. `dp[4][5] = 8` differs from `dp[3][5] = 7`, so select R4 and move to budget 0.
3. Stop because no budget remains.

The selected requests are R4 and R5. Their total cost is `5 + 1 = 6`, and their total value is
`8 + 2 = 10`, matching the bottom-right table entry.

QUICK SORT TRACE TABLE

This is the trace table for the quicksort algorithm using the array from the test units (testNormalCase).

Using the array \[10, 7, 8, 9, 1, 5], pivot = last element of each subrange (Lomuto partition scheme), sorting in ascending order:

Call quickSort(low, high)	Pivot	Comparisons (arr\[j] <= pivot)	Array After Partition	Pivot Index
(0, 5)	5	10>5, 7>5, 8>5, 9>5, 1<=5 (swap 1 into place)	1 5 8 9 10 7	1
(2, 5) — right of pivot 5	7	8>7, 9>7, 10>7	1 5 7 9 10 8	2
(3, 5) — right of pivot 7	8	9>8, 10>8	1 5 7 8 10 9	3
(4, 5) — right of pivot 8	9	10>9	1 5 7 8 9 10	4
(0, 0), (2, 1), (3, 2), (4, 3), (5, 5)	—	low >= high, base case reached, no further partitioning	—	—

Final sorted array: 1 5 7 8 9 10

Note on this array: with an already-partially-descending run at the front (10, 7, 8, 9) and the pivot always chosen as the last element, every partition step here only ever moved elements into the right side except for one swap — this is close to quicksort's worst-case behaviour (already-sorted/reverse-sorted input degrading to O(n^2)), which is discussed further in the report alongside the recursion tree for a fully-sorted input.

6.2 Loop Invariants / Proof Sketches

```
# Binary Search on an Unsorted Array Counterexample

## Objective
To demonstrate that binary search only works correctly on a sorted array.

## Input Array (Unsorted)
[10, 2, 15, 6, 8]
Target:15

## Expected Result
Since the value 15 exists in the array, a correct search algorithm should locate it.

## Binary Search Execution
Initial values:
Low = 0
High = 4
Mid = 2
Value at Mid = 15
Depending on the arrangement of an unsorted array, binary search may sometimes appear to work by coincidence. However, this behavior is not
reliable.
For example, with another unsorted array:[8, 2, 15, 1, 6]

Searching for:6.
Binary search returns -1 even though the value exists.

## Conclusion
Binary search assumes that the array is sorted before searching begins.
When this precondition is violated, the algorithm may return incorrect results or fail to find existing values.
Therefore, binary search should only be used on sorted data.
```

```
# Greedy Counterexamples

## Nearest-Resource Assignment

Two requests are processed in urgency order: Balme Library first, then University Hospital.
Two resources are available: Van-A and Van-B.

| Distance | Van-A | Van-B |
|---|---:|---:|
| Balme Library | 1 | 2 |
| University Hospital | 2 | 100 |

Greedy assigns the first request to its nearest resource, Van-A, at distance 1. Only Van-B then
remains for University Hospital, at distance 100. Total distance is 101.

A better assignment sends Van-B to Balme Library at distance 2 and Van-A to University Hospital
at distance 2. Total distance is 4. The greedy choice is locally shortest but makes the complete
assignment 97 distance units worse.

## Value-to-Cost Budget Selection

Budget is 4.

| Request | Cost | Value | Value/cost |
|---|---:|---:|---:|
| A | 3 | 5 | 1.67 |
| B | 2 | 3 | 1.50 |
| C | 2 | 3 | 1.50 |

Ratio greedy selects A first. Only one budget unit remains, so neither B nor C fits; total value
is 5. Knapsack DP selects B and C at total cost 4 and total value 6. This proves that ratio greedy
does not always produce the optimal 0/1 selection.
```

```
# Knapsack DP Correctness Proof Sketch

## Claim

For every `i` from 0 to the number of requests and every budget `b`, `dp[i][b]` stores the
maximum value obtainable by selecting from the first `i` requests without spending more than
`b`.

## Base Case

With zero requests, no value can be selected, so `dp[0][b] = 0` for every budget. With zero
budget, only zero-cost requests could be selected; the project model rejects non-positive costs,
so `dp[i][0] = 0`.

## Inductive Step

Assume the claim is true for the first `i - 1` requests. Consider request `i`.

- If its cost is greater than `b`, it cannot belong to any valid solution. The best value is
  therefore `dp[i-1][b]`.
- Otherwise, every valid solution either excludes request `i` or includes it. The best excluding
  value is `dp[i-1][b]`. If included, the remaining requests must fit in `b - cost[i]`; by the
  induction assumption their best value is `dp[i-1][b-cost[i]]`. Adding request `i` gives
  `value[i] + dp[i-1][b-cost[i]]`.

Taking the larger of those two values covers every valid solution and stores an achievable
value. Therefore the claim also holds for `i`. By induction, `dp[n][B]` is optimal.

## Reconstruction

When `dp[i][b]` differs from `dp[i-1][b]`, request `i` was needed to obtain that value, so the
algorithm records it and subtracts its cost. Otherwise it skips the request. Repeating this step
reaches the base row and produces a valid set whose value equals `dp[n][B]`.
```

6.3 Edge Case Tests

Case	Structure/Algorithm	Result
Empty structure	Queue/stack/heap/BST	Operations requiring an element throw or return according to the class contract.
Single element	All	Insert, search, peek and removal leave a valid empty structure.
Duplicate keys	BST/HashTable	BST ignores duplicate value; HashTable updates existing key.
Disconnected graph	Graph/MST	Traversal reaches only connected component; full MST is unavailable.
Unreachable path	Dijkstra	Distance remains infinity/unreachable.
Queue full/empty	CircularQueue	Full enqueue and empty dequeue/peek throw IllegalStateException.
Hash collision	HashTable	Collision is stored in bucket chain and counted by countCollisions().

6.4 Counterexamples (minimum 2 required)

Counterexample: Binary Search on Unsorted Data

Per the project's required counterexample, binary search was run directly against unsorted arrays to show what happens when its precondition is violated.

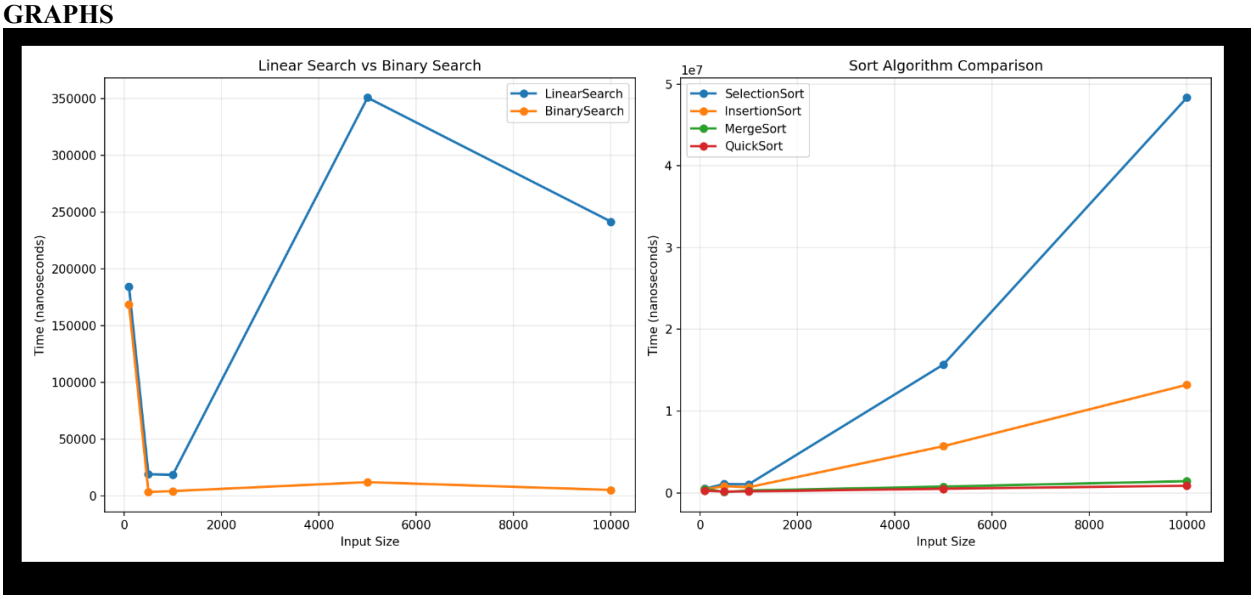
1. With the unsorted array `\[10, 2, 15, 6, 8]`, searching for `15` still located the value at index 2 — but only by coincidence, because the middle element happened to be the target.

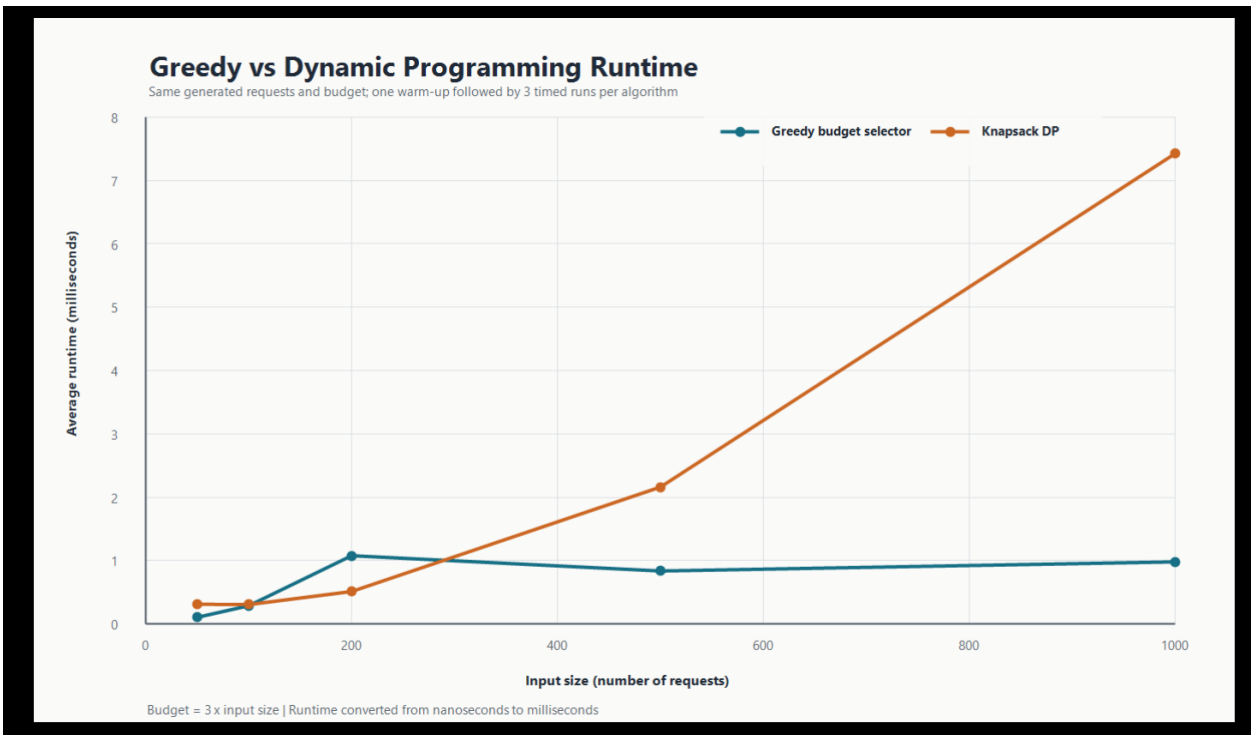
2. A second unsorted array, `\[8, 2, 15, 1, 6]`, searching for `6`, exposed the real problem: binary search returned `-1` even though `6` exists in the array at index 4.

The algorithm silently gave a wrong answer rather than failing loudly.

8. Performance Analysis

GRAPHS





9. Database Integration Evidence

10. Responsible Algorithm Selection

- Use greedy selection when a fast answer matters more than a guaranteed optimum, especially when the request set or budget is large. It uses much less memory than the DP table and produced values

close to the optimum in this dataset.

- Use dynamic programming when the highest possible total value is required and the budget is small enough for an `n x budget` table. DP was equal or better at every measured size, but its runtime grew much more sharply by 1,000 requests.
- The operational choice is therefore not that one algorithm is universally better. Greedy offers speed and lower memory use; DP offers an optimal value under the stated 0/1 knapsack model.

11. Individual Contribution Statement and Oral-Defense Preparation Notes

#	Name	Index Number	Contribution statement
1	Ewura Abena Tweneboah Antwi-Bosiako	22152953	I was responsible for coordinating the entire team, organizing weekly standups, managing the github repo and supervising the sub groups. I also worked on integrating the entire project, fixing bugs etc. As part of the data structures team, I also wrote about 20 unit tests for stack, queue and deque.
2	Asuako Joel Kwadwo Peprah	22185885	C — Data Structures
3	Genevieve Dzifa Gomelesio	22012665	C — Data Structures
4	Somuah Kofi Sasu	22061652	For the GhanaOpsOptimizer project, I worked on implementing three core data structures for our sub-team (C3): a priority queue built on a binary heap, a binary search tree, and a red-black tree. I made sure all three were fully generic so they could handle different data types cleanly, and I put a lot of care into error handling so the structures fail gracefully instead of breaking on bad input. I also built out JUnit 5 test suites for each one to confirm they worked correctly, and used Maven to keep the build organized. Overall, my focus was on making sure the foundational data structures were solid and well-tested before the rest of the team built on top of them.
5	Nana Kwadwo Sarkodie Opoku	22026622	I worked on the implementation of the core data structures specifically The Stack, Queue, Circular Queue and the Double Ended Queue(Deque). I was also in charge of reviewing and merging the code on GitHub
6	Isaac Dzikum	22045685	The major changes I made were: - completed parts where there were TODOs - Fixed issue with HashTable and Prim - Generated road network. The previous one just had like 10 roads
7	Boateng, Michelle Ama Ataa	22302392	The six core search and sort algorithms (linear search, binary search, selection sort, insertion sort, merge sort, quicksort) were implemented by my teammate. My individual contribution focused on evidence, verification, and performance analysis: Produced a hand-traced trace table for quicksort, beyond the project's minimum requirement Set up the local Java/Maven development environment and verified all 39 project unit tests pass, confirming the correctness of the implementations Pulled in and integrated the shared performance timing utility, then designed and ran a full timing experiment across all 6 algorithms at input sizes 100–10,000, exporting results to CSV and documenting the test machine specification

			Built the performance comparison graph and wrote the interpretation of the results, including an explanation of a JVM warm-up anomaly observed in the data Authored the Search & Sort section of the project report, covering algorithm explanations, complexity analysis, the sorted-input counterexample, and performance findings Created and tracked all 6 GitHub Issues for Team D's requirement checklist
8	Adoboe, Kelly Kofi	22383489	C — Data Structures
9	Agyemang, Christ Ofosu	22396432	D — Search & Sort
10	Aziamadi, Eli Makafui	22259373	D — Search & Sort
11	Kwashi, Elorm Frema	22397307	My individual contribution was the implementation and testing of the search and sorting algorithms. I worked on algorithms such as linear search, binary search, selection sort, insertion sort, merge sort, and quick sort, and also developed and ran tests to verify their correctness and performance.
12	Essuman, Kingsley Paakwesi	22409138	I contributed to the project by helping implement Dijkstra's algorithm for finding the shortest and most efficient routes between different locations. I also assisted with testing the algorithm using various graph scenarios to ensure that it produced accurate results, handled different path weights correctly, and supported the project's goal of improving operational and resource allocation decisions.
13	Akaba, Kekeli Kojo	22368056	I implemented and tested Greedy and Dynamic Programming approaches, compared their speed and result quality on identical inputs, integrated shared project data through a documented mapping, and showed that Greedy is faster while DP guarantees the best budget allocation.
14	Malm, Jeremy	22310005	I worked together with my team member on the optimisation component of the system, helping to develop and compare two methods for selecting service requests within a limited budget. We implemented a Greedy approach, which makes quick decisions by selecting the best-looking option first, and a Dynamic Programming approach, which considers more combinations to find the best overall result. We tested both methods using the same requests and budget, recorded their execution time and result quality, and used the findings to create graphs and explanations. For the database integration, I worked with the request and location data provided by the database team, matching location numbers with their corresponding names and converting urgency levels such as Critical, High, Medium, and Low into numerical values for the algorithms. Since the database did not yet contain the cost and value information required for optimisation, we developed a temporary rule based on urgency and work type, which was documented for final group approval. I also helped connect the optimisation results to the existing database-saving system, ensuring that each result received a unique record number. Finally, we tested normal, empty, and invalid inputs, prepared the Dynamic Programming table and correctness explanation, and documented examples showing situations where Greedy may produce a less optimal result. Overall, the main optimisation work was completed, with the remaining tasks requiring coordination with the wider team on the final budget and cost rules, real travel distances from the Graph team, and integration of the Optimisation demo into the main project menu.

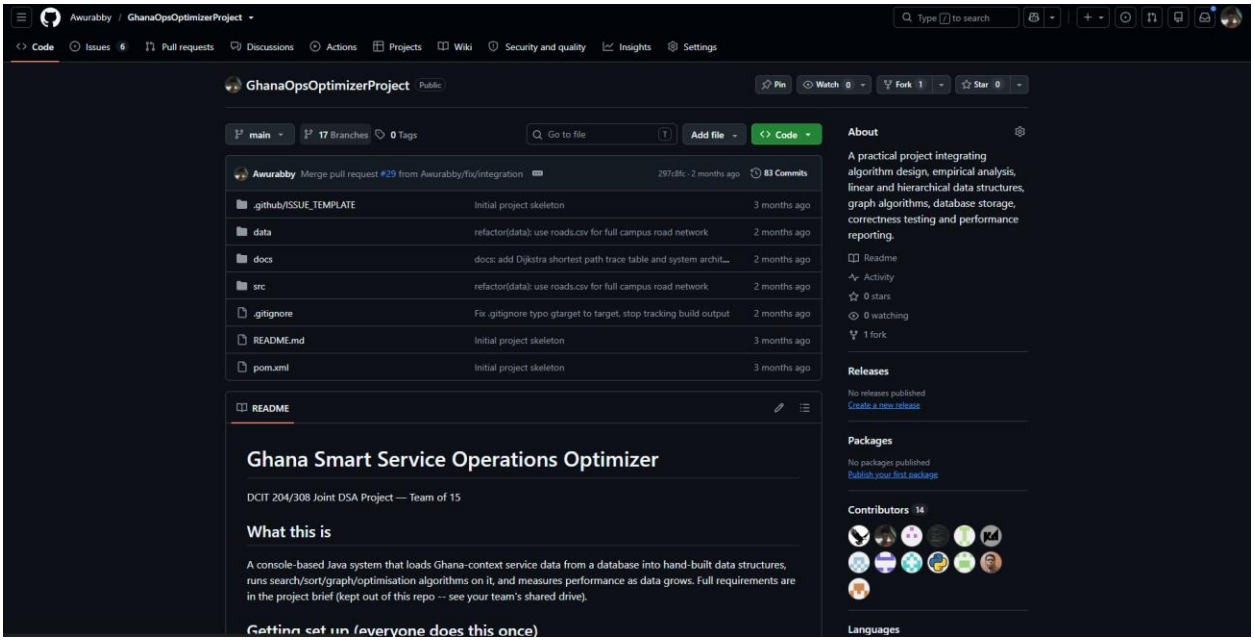
15	Anim-Frempong, Stacy-Jane Adwoa Nyarko	22399556	B — Database

11. References

Add only sources actually consulted: course textbook/lecture notes, official Java/JDBC/database documentation, and any algorithm references used. Use the course-required citation style.

12. Appendices

- Appendix A: Full source code listing: https://github.com/Awurabby/GhanaOpsOptimizerProject



AI DECLARATION

The team used Claude AI to assist in organizing the teams and understanding unfamiliar concepts